

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ BELHASSAN
Nom d'usage ▶ Entrez votre nom d'usage ici.
Prénom ▶ Salah
Adresse ▶ 8 Avenue de Canto Perdix, 13500 Martigues

Titre professionnel visé

Développeur Web et Web Mobile
CCP Développer la partie back-end d'une application web ou web mobile
sécurisée

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie back-end d'une application web ou web mobile sécurisée p. 5

- ▶ Mettre en place une base de données relationnellep. p. 5
- ▶ Développer des composants d'accès aux données SQL et NoSQL.....p. p. 7
- ▶ Développer des composants métier coté serveurp p. 10
- ▶ Documenter le déploiement d'une application dynamique web ou web mobilep p. 16

Titres, diplômes, CQP, attestations de formation *(facultatif)* p. 18

Déclaration sur l'honneur p. 19

Documents illustrant la pratique professionnelle *(facultatif)* p. 20

Annexes *(Si le RC le prévoit)* p. 21

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°1 ► Mettre en place une base de données relationnelle

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur le projet MarsIA, plateforme de référencement de courtes vidéos sur l'intelligence artificielle, j'ai pris en charge la conception de la base de données.

J'ai identifié six entités principales : users, filmmakers, videos, roles, social_networks et jury_reviews. Un réalisateur peut fournir plusieurs vidéos et posséder plusieurs comptes sur les réseaux sociaux, un utilisateur se voit attribuer un rôle, et chaque vidéo peut recevoir plusieurs avis de jury, chacun relié à la fois à la vidéo et à l'utilisateur qui l'a rédigé.

À partir de ce schéma, j'ai défini les tables correspondantes en précisant les types de données, les clés primaires et étrangères, ainsi que les cardinalités entre chaque entité (majoritairement 0,n et 0,1).

← MCD : Nouveau MCD_0'

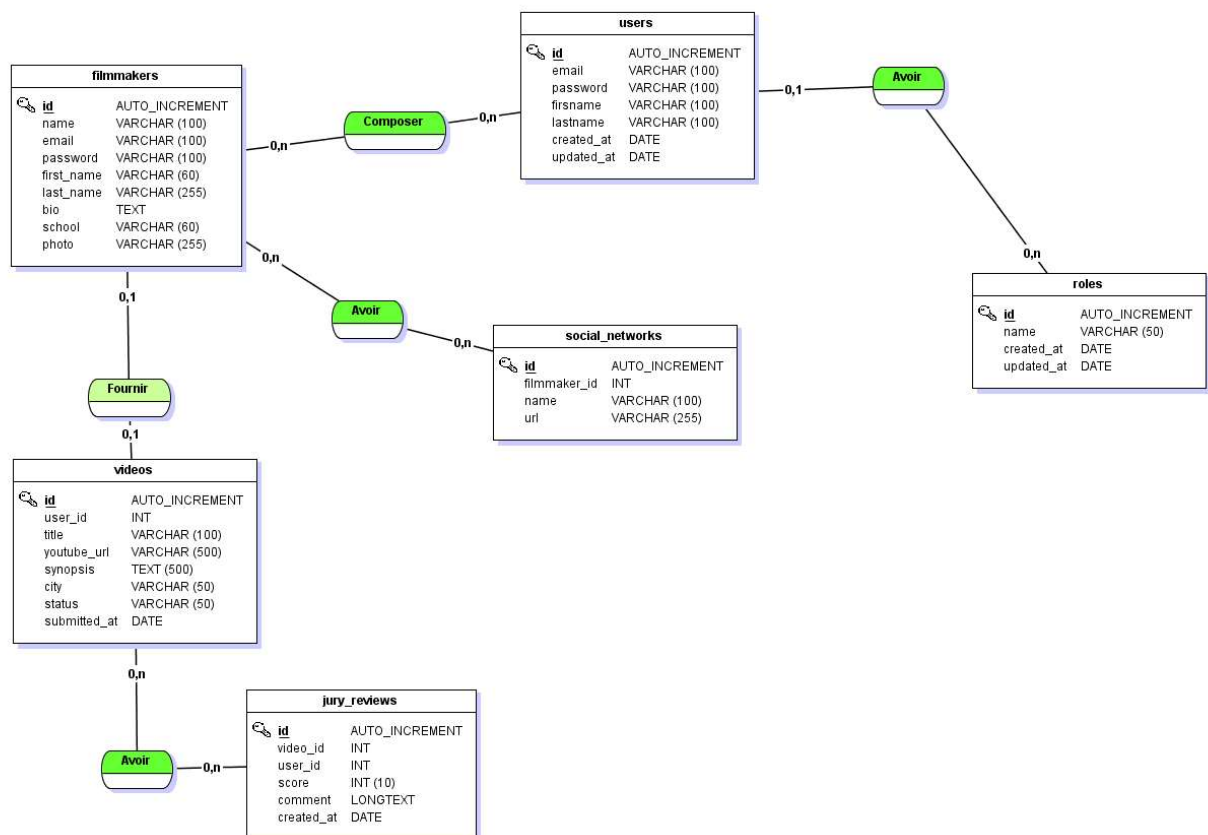


Figure 1 : Schéma CMD

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

JMerise outil portable de modélisation de MCD et génération du MLD
MySQL et MySQL Workbench
Clés primaires, clés étrangères, contraintes d'intégrité

3. Avec qui avez-vous travaillé ?

J'ai travaillé dans un groupe de 5 personnes avec la supervision de mes formateurs

4. Contexte

Nom de l'entreprise, organisme ou association ► *La Plateforme*

Chantier, atelier, service ► Projet MarsIA

Période d'exercice ► Du 05/01/2026 au 16/01/2026

5. Informations complémentaires (facultatif)

Cliquez ici pour taper du texte.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°2 ▶ Développer des composants d'accès aux données SQL et NoSQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur mon projet personnel de gestion de tournois de tennis, j'ai développé la couche d'accès aux données pour les entités Player et Tournament.

J'ai mappé les entités JPA (PlayerEntity, TournamentEntity) sur les tables PostgreSQL correspondantes, en définissant les contraintes de colonnes (longueur, unicité, nullabilité) et la relation many-to-many entre joueurs et tournois via une table de jointure player_tournament.

J'ai ensuite créé les interfaces PlayerRepository et TournamentRepository en étendant JpaRepository, avec des méthodes dérivées (query methods) comme findOneByIdentifiant ou findOneByFirstNameIgnoreCaseAndLastNameIgnoreCaseAndBirthDate, pour éviter d'écrire du SQL manuel sur les cas de lecture courants.

Pour découvrir le volet NoSQL, j'ai mené un exercice pratique autour d'une base de petites annonces sous MongoDB. J'ai déployé une instance MongoDB dans un conteneur Docker, ajouté au docker-compose d'un exercice existant, puis initialisé la collection avec un script mongosh (insertMany) avant d'explorer les données avec MongoDB Compass.

Chaque annonce est un document JSON dont le sous-document details varie selon la catégorie — kilométrage pour un véhicule, surface pour un appartement — ce qui illustre la souplesse de schéma du modèle documentaire par rapport au relationnel.

J'ai ensuite manipulé l'ensemble des opérations CRUD sur cette collection : lectures filtrées (find avec \$lte et \$gte, findOne), insertions (insertOne, insertMany), mises à jour ciblées ou groupées avec \$set (modification d'un prix, ajout d'un champ vendu, remise appliquée à toutes les annonces d'une ville), et suppressions (deleteOne, deleteMany par critère).

```
@Repository 10 usages
public interface PlayerRepository extends JpaRepository<PlayerEntity, Long> {
    Optional<PlayerEntity> findOneByIdentifiant(UUID identifiant); 8 usages

    Optional<PlayerEntity> findOneByFirstNameIgnoreCaseAndLastNameIgnoreCaseAndBirthDate(
        String firstName,
        String lastName,
        LocalDate birthDate
    );
}
```

Figure 2 : Extrait du fichier "PlayerRepository.java"

```
@Entity
@Table(name = "player", schema = "public")
public class PlayerEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    @Column(name = "last_name", length = 50, nullable = false) 4 us
    private String lastName;

    @Column(name = "first_name", length = 50, nullable = false) 4 us
    private String firstName;

    @Column(name = "identifier", nullable = false, unique = true) 3
    private UUID identifier;

    @Column(name = "birth_date", nullable = false) 4 usages
    private LocalDate birthDate;

    @Column(name = "points", nullable = false) 4 usages
    private Integer points;

    @Column(name = "rank", nullable = false) 4 usages
    private Integer rank;
```

Figure 3 : extrait de l'entité "PlayerEntity"

```
@Repository 10 usages
public interface TournamentRepository extends JpaRepository<TournamentEntity, Long> {
    Optional<TournamentEntity> findOneByIdentifier(UUID identifier); 8 usages

    Optional<TournamentEntity> findOneByNameIgnoreCase(String name); 2 usages
}
```

Figure 4 : TournamentRepository


```
@Table(name = "tournament", schema = "public")
public class TournamentEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    @Column(name = "identifier", nullable = false, unique = true) 3 usages
    private UUID identifier;

    @Column(name = "name", length = 100, nullable = false) 3 usages
    private String name;

    @Column(name = "start_date", nullable = false) 3 usages
    private LocalDate startDate;

    @Column(name = "end_date", nullable = false) 3 usages
    private LocalDate endDate;

    @Column(name = "prize_money", nullable = true) 3 usages
    private Integer prizeMoney;

    @Column(name = "capacity", nullable = false) 4 usages
    private Integer capacity;

    @ManyToMany(mappedBy = "tournaments", fetch = FetchType.EAGER) 4 usages
    private Set<PlayerEntity> players = new HashSet<>();

    public TournamentEntity() {
    }

    public TournamentEntity(UUID identifier, String name, LocalDate startDate, LocalDate endDate, Integer prizeMoney, Integer capacity) {
        this.identifier = identifier;
        this.name = name;
        this.startDate = startDate;
        this.endDate = endDate;
        this.prizeMoney = prizeMoney;
        this.capacity = capacity;
    }

    public Long getId() { return id; }

    public UUID getIdentifier() { return identifier; }

    public void setIdentifier(UUID identifier) { this.identifier = identifier; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }
```

Figure 5 : extrait de l'entité "TournamentEntity"

```
CREATE TABLE player_tournament
(
    player_id bigint NOT NULL,
    tournament_id bigint NOT NULL,
    CONSTRAINT player_tournament_pkey PRIMARY KEY (player_id, tournament_id),
    CONSTRAINT fk_player FOREIGN KEY (player_id) REFERENCES public.player (id),
    CONSTRAINT fk_tournament FOREIGN KEY (tournament_id) REFERENCES public.tournament (id)
);
```

Figure 6 : script SQL pour la création de la table "player_tournament"

2. Précisez les moyens utilisés :

Spring Data JPA, PostgreSQL, Flyway (scripts de migration versionnés pour créer les tables et les données de référence), IntelliJ IDEA.

3. Avec qui avez-vous travaillé ?

Projet personnel, réalisé seul, en autoformation.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Projet personnel*

DOSSIER PROFESSIONNEL (DP)

Chantier, atelier, service ▶ Application de gestion de tournois de tennis
Période d'exercice ▶ Du 15/06/2026 au 19/07/2026

5. Informations complémentaires (facultatif)

Cliquez ici pour taper du texte.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°3 ▶ Développer des composants métier coté serveur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai développé la couche service qui porte la logique métier de l'application, séparée de la couche d'accès aux données et de la couche web.

Le PlayerService centralise les règles métier liées aux joueurs : vérifier qu'un joueur n'existe pas déjà (même prénom, nom et date de naissance) avant de le créer, lever une exception métier dédiée (PlayerNotFoundException, PlayerAlreadyExistsException) plutôt que de laisser remonter une erreur technique, et recalculer automatiquement le classement de tous les joueurs après chaque création, mise à jour ou suppression.

Pour ce recalcul, j'ai isolé la règle de tri dans une classe dédiée, RankingCalculator, qui trie les joueurs par points décroissants et réattribue les rangs — ce qui garde PlayerService lisible et rend cette règle testable indépendamment.

J'ai appliqué la même logique pour les tournois avec RegistrationService, qui gère l'inscription d'un joueur à un tournoi en enchaînant plusieurs contrôles métier : le tournoi existe, il n'est pas complet, le joueur existe, il n'est pas déjà inscrit.

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL (DP)

Spring (@Service, injection de dépendances par constructeur), gestion d'exceptions métier personnalisées, journalisation avec SLF4J, tests unitaires et d'intégration (JUnit, Mockito) pour valider les règles avant mise en production.

3. Avec qui avez-vous travaillé ?

Cliquez ici pour taper du texte.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Projet personnel*

Chantier, atelier, service ► Application de gestion de tournois de tennis

Période d'exercice ► Du 15/06/2026 au 19/07/2026

5. Informations complémentaires (facultatif)

```
public Player create(PlayerToCreate playerToCreate) {
    log.info("Invoking create with playerToCreate={}", playerToCreate);

    try {
        Optional<PlayerEntity> player = playerRepository.findOneByFirstNameIgnoreCaseAndLastNameIgnoreCaseAndBirthDate(
            playerToCreate.firstName(), playerToCreate.lastName(), playerToCreate.birthDate());
        if (player.isPresent()) {
            log.warn("Player to create with firstName={} lastName={} and birthDate={} already exists",
                playerToCreate.firstName(), playerToCreate.lastName(), playerToCreate.birthDate());
            throw new PlayerAlreadyExistsException(playerToCreate.firstName(), playerToCreate.lastName(), playerToCreate.birthDate());
        }

        PlayerEntity playerToRegister = new PlayerEntity(
            playerToCreate.lastName(),
            playerToCreate.firstName(),
            UUID.randomUUID(),
            playerToCreate.birthDate(),
            playerToCreate.points(),
            rank: 999999999);

        PlayerEntity registeredPlayer = playerRepository.save(playerToRegister);

        RankingCalculator rankingCalculator = new RankingCalculator(playerRepository.findAll());
        List<PlayerEntity> newRanking = rankingCalculator.getNewPlayersRanking();
        playerRepository.saveAll(newRanking);

        return this.getByIdentifier(registeredPlayer.getIdentifier());
    } catch (DataAccessException e) {
        log.error("Could not create player={}", playerToCreate, e);
        throw new PlayerDataRetrievalException(e);
    }
}
```

Figure 7 : Extrait du fichier "PlayerService.java"

```
@Service 7 usages
public class RegistrationService {

    private final Logger log = LoggerFactory.getLogger(RegistrationService.class); 4 usages

    @Autowired
    private final TournamentRepository tournamentRepository;

    @Autowired
    private final PlayerRepository playerRepository;

    public RegistrationService(TournamentRepository tournamentRepository, PlayerRepository playerRepository) {
        this.tournamentRepository = tournamentRepository;
        this.playerRepository = playerRepository;
    }

    public void register(UUID tournamentIdentifier, UUID playerToRegister) { 7 usages
        Optional<TournamentEntity> existingTournament = tournamentRepository.findOneByIdentifier(tournamentIdentifier);
        if (existingTournament.isEmpty()) {
            log.warn("Could not find tournament {} to register player", tournamentIdentifier);
            throw new TournamentRegistrationException("Tournament " + tournamentIdentifier + " does not exist");
        }

        if (existingTournament.get().isFull()) {
            log.warn("Tournament {} is full", existingTournament.get().getIdentificier());
            throw new TournamentRegistrationException("Tournament " + existingTournament.get().getIdentificier() + " is full");
        }

        Optional<PlayerEntity> existingPlayer = playerRepository.findOneByIdentifier(playerToRegister);
        if (existingPlayer.isEmpty()) {
            log.warn("Could not find player {} to register", playerToRegister);
            throw new TournamentRegistrationException("Player " + playerToRegister + " does not exist");
        }

        if (existingTournament.get().hasPlayer(existingPlayer.get())) {
            log.warn("Player {} is already registered to tournament {}", playerToRegister, tournamentIdentifier);
            throw new TournamentRegistrationException("Player " + playerToRegister + " is already registered to tournament " + tournamentIdentifier);
        }

        existingPlayer.get().addTournament(existingTournament.get());
        playerRepository.save(existingPlayer.get());
    }
}
```

Figure 8 : Extrait du fichier "RegistrationService.java"

```
public class RankingCalculator { 6 usages

    private final List<PlayerEntity> currentPlayersRanking; 4 usages

    public RankingCalculator(List<PlayerEntity> currentPlayersRanking) { 3 usages
        this.currentPlayersRanking = currentPlayersRanking;
    }

    public List<PlayerEntity> getNewPlayersRanking() { 3 usages
        currentPlayersRanking.sort(( PlayerEntity player1, PlayerEntity player2) -> Integer.compare(player2.getPoints(), player1.getPoints()));

        List<PlayerEntity> updatedPlayers = new ArrayList<>();
        for (int i = 0; i < currentPlayersRanking.size(); i++) {
            PlayerEntity updatedPlayer = currentPlayersRanking.get(i);
            updatedPlayer.setRank(i + 1);
            updatedPlayers.add(updatedPlayer);
        }
        return updatedPlayers;
    }
}
```

Figure 9 : Extrait du fichier "RankingCalculator.java"

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°4 ▶

Documenter le déploiement d'une application dynamique web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai préparé le déploiement de l'application en containerisant le back-end avec Docker et en séparant les configurations selon l'environnement d'exécution.

J'ai écrit un Dockerfile en multi-stage build : une première étape compile le projet avec Maven (profil prod) dans une image contenant le JDK, puis une seconde étape ne conserve que le .jar généré dans une image JRE plus légère, exécutée par un utilisateur dédié non-root pour limiter les privilèges du conteneur.

Côté configuration, j'ai séparé les profils Spring (application.properties, application-dev.properties, application-prod.properties) : les identifiants de connexion à la base en production sont injectés à l'extérieur du code plutôt qu'écrits en dur, et Flyway applique automatiquement au démarrage les scripts de migration versionnés (db/migrations/common puis db/migrations/prod) pour créer et faire évoluer le schéma de la base de données de façon reproductible.

2. Précisez les moyens utilisés :

Docker (multi-stage build), Maven (profils dev/prod), Spring Boot profiles, Flyway pour les migrations de base de données, Spring Boot Actuator pour exposer un endpoint de santé (/health) utilisable pour vérifier que le conteneur déployé répond correctement.

DOSSIER PROFESSIONNEL (DP)

3. Avec qui avez-vous travaillé ?

Cliquez ici pour taper du texte.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Projet personnel*

Chantier, atelier, service ► Application de gestion de tournois de tennis

Période d'exercice ► Du 15/06/2026 au 19/07/2026

5. Informations complémentaires (facultatif)

Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL (DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussigné(e) Belhassan Salah, ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à *Martigues*..... le *19/07/2026*.....

pour faire valoir ce que de droit.

Signature :



DOSSIER PROFESSIONNEL (DP)

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL (DP)

ANNEXES

(Si le RC le prévoit)

